

Phase 5 Report

Kubernetes Workload Analyzer — React dashboard, single-binary serving

Phase status	DONE (2026-09-26)
Report date	2026-09-26
Repository	github.com/ISHANT57/kubernetes-workload-analyzer
State of main	7d3455a , merged as 1cb1cd1 (PR #10)
Stack	React 19, Vite 8, TypeScript, uPlot, react-router-dom 7 (D-002)
Written for	the project owner, as a study and reference document

Evidence labels used throughout. **MEASURED** a command was run on this machine and the number is its output · **VERIFIED** checked against upstream documentation, source or a registry · **ESTIMATED** a projection, not measured · **UNVERIFIED** not yet tested; a test is named.

Contents

1. What Phase 5 was for
2. Pages built
3. Sticky-on-failure polling
4. New backend surface: /api/timeseries
5. Single-binary serving
6. TypeScript build issues found and fixed
7. Live Playwright verification
8. A real mobile CSS bug found and fixed
9. Git and GitHub workflow
10. Current state and operating
11. Open items and next phase

1. What Phase 5 was for

Phase 4 produced a findings API with no UI consumer. Phase 5 built the dashboard decided in D-002: React + Vite + TypeScript, built to static files served by the Go backend itself (no Node server at runtime), uPlot for the charts that answer a specific question. Done condition (`PHASES.md`): Overview, Findings, Detail, Workloads, Platform Status pages.

2. Pages built

Page	Purpose
Overview	top findings, ranked health-first then resource then cost
Findings	full filterable list (severity, category, rule)
Finding Detail	full evidence table, threshold, window, data-quality/confidence badge, caveats, and — for R001/R002 — a real uPlot usage-vs-request chart
Workloads	every discovered workload, not only ones with findings
Platform Status	the analyzer's own health: last run status, query errors, data coverage

uPlot is used imperatively (a ref + `useEffect`), not through a React wrapper component — kept as a thin, explicit integration rather than adding a charting-library dependency wrapper of its own.

3. Sticky-on-failure polling

`src/hooks/usePolling.ts` keeps the last-good data on screen and marks it `partial/stale` rather than blanking the UI on a transient API failure — the same sticky-snapshot pattern already used on the backend (Phase 3's runner, Phase 4's findings), now carried into the frontend so a brief Prometheus or network hiccup does not flash the dashboard empty.

4. New backend surface: `/api/timeseries`

The only backend change in this phase: `GET /api/timeseries`, reusing the evidence builder's existing, fixed, parameterized Prometheus queries (`CPUUsageSeries` / `MemoryUsageSeries` in `internal/evidence/builder.go`) — the browser still never sends PromQL, only `namespace/workload/container/ metric/window` query parameters. Response shape: `{"points": [{ "t": unix_seconds, "v": value}, ...]}`.

5. Single-binary serving

The Go backend now also serves the built frontend directly: `STATIC_DIR` env var + SPA fallback (`internal/httpserver's spaHandler`), so the whole product is one deployable binary with no separate Node server at runtime — the tradeoff D-002 was decided around. Without `STATIC_DIR` the backend still works as an API-only server, used during frontend development where `npm run dev` proxies to it instead.

6. TypeScript build issues found and fixed

Constructor parameter-property shorthand rejected. `erasableSyntaxOnly` disallows `constructor(public status: number);` `ApiError` was rewritten with an explicit field plus a constructor body.

Unused parameter build error. `noUnusedParameters` caught an unused `metric` parameter in `formatEvidenceValue();` removed.

7. Live Playwright verification

Verified with a real backend and real cluster data via Playwright browser automation, not just built and assumed to work:

- Desktop viewport — zero console errors.
- Mobile viewport (390×844) — zero console errors.
- Dark mode (`prefers-color-scheme` emulation) — zero console errors.

8. A real mobile CSS bug found and fixed

Long PromQL query text in the evidence table overflowed the card layout at 390px width — only visible by actually rendering the page at mobile size, not from reading the code. Fixed with `word-break: break-all` on `.evidence-table` code and `overflow-x: auto` on `.card`, then re-verified at 390px with a fresh screenshot.

9. Git and GitHub workflow

Branch `phase-5` → commit `7d3455a` ("feat(phase-5): React dashboard, /api/timeseries, single-binary static serving") → PR #10 → merged as `1cb1cd1` → branch deleted.

10. Current state and operating

```
cd frontend && npm install && npm run build  
cd backend && STATIC_DIR=$(pwd)/../frontend/dist go run ./cmd/analyzer  
# open http://localhost:8080/
```

11. Open items and next phase

- Resource findings show no way to jump into raw Prometheus/Grafana time series yet — Phase 6.
- No in-cluster deployment yet — the backend and frontend build still run on the host — Phase 7.